MODULE 2: CLINICAL BIOSTATISTICS FOR KIDNEY DISEASE RESEARCH

Darya Y

2026-06-07

```
# ============================================================================
# MODULE 2: CLINICAL BIOSTATISTICS FOR KIDNEY DISEASE RESEARCH
# ============================================================================
#
# INTRODUCTION: WHAT IS CLINICAL BIOSTATISTICS IN THIS CONTEXT?
```

```
# A kidney disease research collaboration.
# The data looks like this:
#   - Patients with Chronic Kidney Disease (CKD), recruited at a clinic
#   - Measurements taken at multiple time points (e.g., baseline, 6 months, 12 months)
#   - Lab values: eGFR (estimated Glomerular Filtration Rate) - the gold standard
#     measure of kidney function. Normal is ~90+. Below 60 = CKD. Below 15 = failure.
#   - Immune phenotyping from flow cytometry: monocyte scores, T cell scores
#     These tell us about the inflammatory state of the patient.
#
# THE CORE QUESTION:
#   "Can we predict which CKD patients will progress to kidney failure based on
#    their immune cell profiles measured early in the disease course?"
#
# WHY THIS MATTERS:
#   - If monocyte activation (a marker of chronic inflammation) predicts faster
#     eGFR decline, clinicians could intensify immunosuppression earlier
#   - This is a real research question - innate immune cells are implicated in
#     CKD progression through tubulointerstitial fibrosis
#
# STATISTICAL CHALLENGES IN THIS DATA:
#   1. REPEATED MEASURES: Same patient at 3 time points → not independent
#   2. MISSING DATA: Patients drop out (often the sickest ones - informative censoring)
#   3. CONFOUNDING: Age, sex, diabetes status all affect eGFR
#   4. MULTIPLE TESTING: If we test 50 immune markers, ~2.5 will be significant by chance
#
# This module teaches how to handle all four challenges rigorously.
# ============================================================================


# ============================================================================
# SECTION 0: PACKAGE INSTALLATION & LOADING
# ============================================================================
# Check for each package before attempting install.
# suppressMessages() reduces noise during loading - useful in production scripts.
```

```r
required_packages <- c("dplyr", "ggplot2", "lme4", "lmerTest", "survival",
                       "broom", "broom.mixed", "tidyr", "tidyverse", "ggpubr",
                       "ggfortify", "patchwork", "scales", "RColorBrewer")

for (pkg in required_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    message(paste("Installing:", pkg))
    install.packages(pkg, repos = "https://cloud.r-project.org", quiet = TRUE)
  }
}

suppressMessages({
  library(dplyr)
  library(ggplot2)
  library(lme4)
  library(lmerTest)   # Adds p-values to lme4 output via Satterthwaite approximation
  library(survival)
  library(broom)
  library(broom.mixed) # tidy() for mixed effects models
  library(tidyr)
  library(patchwork)   # Combine ggplot panels
  library(scales)
  library(RColorBrewer)
  library(tidyverse)
  library(ggpubr)
  library(ggfortify)
})

# Create output directory for plots
plot_dir <- "module2_plots"
dir.create(plot_dir, showWarnings = FALSE, recursive = TRUE)

cat("=== MODULE 2: Clinical Biostatistics ===\n")
```

```
## === MODULE 2: Clinical Biostatistics ===
```

```r
cat("Output plots will be saved to:", plot_dir, "\n\n")
```

```
## Output plots will be saved to: module2_plots
```

```r
# ============================================================================
# SECTION 1: SIMULATE REALISTIC PATIENT DATA
# ============================================================================
# WHY SIMULATE? In research, we often need to:
#   - Test the analysis pipeline before real data arrives with examples that mirror real data structure
#   - Demonstrate statistical power (simulate → analyze → check if we detect effects)
#
# KEY DESIGN DECISIONS IN THIS SIMULATION:
#   - 120 patients: 80 CKD + 40 Healthy controls (realistic 2:1 ratio)
#   - 3 time points: baseline (0), 6 months, 12 months
#   - eGFR: CKD patients decline ~2-3 mL/min/1.73m² per 6 months on average
#     (published literature on CKD progression rates)
#   - Immune scores: monocyte_score elevated in CKD, and HIGHER scores correlate
#     with FASTER decline (the hypothesis)
#   - Random effects: each patient has their own baseline eGFR and their own
```

```r
#     rate of decline (some progress fast, some slow - biological variability)

set.seed(42)  # For reproducibility

n_ckd     <- 80   # CKD patients
n_healthy <- 40   # Healthy controls
n_total   <- n_ckd + n_healthy
time_points <- c(0, 6, 12)  # months

# --- Simulate patient-level (baseline) characteristics ---
# Fixed properties of each patient, measured once

patient_data <- data.frame(
  patient_id = paste0("PT", sprintf("%03d", 1:n_total)),
  condition  = c(rep("CKD", n_ckd), rep("Healthy", n_healthy)),

  # Age: CKD patients tend to be older (mean 62) vs healthy controls (mean 48)
  # rnorm(n, mean, sd) - normal distribution
  age = c(round(rnorm(n_ckd, mean = 62, sd = 12)),
          round(rnorm(n_healthy, mean = 48, sd = 15))),

  # Sex: roughly 55% male in CKD cohorts (CKD is slightly male-predominant)
  sex = c(sample(c("Male", "Female"), n_ckd, replace = TRUE, prob = c(0.55, 0.45)),
          sample(c("Male", "Female"), n_healthy, replace = TRUE, prob = c(0.50, 0.50))),

  # Baseline eGFR:
  #   CKD: mean 52, SD 12 (Stage 3 CKD range: 30-59)
  #   Healthy: mean 88, SD 10 (normal range: 60-120)
  eGFR_baseline = c(pmax(25, rnorm(n_ckd, mean = 52, sd = 12)),  # pmax prevents unrealistic negative v
                    pmin(120, pmax(60, rnorm(n_healthy, mean = 88, sd = 10)))), # pmax prevents unreali

  # Monocyte score from flow cytometry (arbitrary units, higher = more activated)
  # CKD patients have chronically activated monocytes (NF-kB signalling, IL-6 production)
  monocyte_score = c(rnorm(n_ckd, mean = 0.65, sd = 0.18),
                     rnorm(n_healthy, mean = 0.30, sd = 0.12)),

  # T cell score: CKD also affects T cell exhaustion
  t_cell_score = c(rnorm(n_ckd, mean = 0.45, sd = 0.15),
                   rnorm(n_healthy, mean = 0.55, sd = 0.12)),

  # Individual rate of eGFR decline (random effect):
  # Each patient has their own "speed" of disease progression
  # Mean decline: -2.5 mL/min per 6 months for CKD; ~0 for healthy
  # SD 1.5: some patients progress much faster (rapid progressors)
  decline_rate = c(rnorm(n_ckd, mean = -2.5, sd = 1.5),
                   rnorm(n_healthy, mean = 0, sd = 0.3)),

  stringsAsFactors = FALSE # Keep strings as is. Do not convert it into a categorical Factor.
)

# Ensure monocyte scores are bounded (0, 1) - these represent proportions
patient_data$monocyte_score <- pmin(1, pmax(0, patient_data$monocyte_score))
patient_data$t_cell_score   <- pmin(1, pmax(0, patient_data$t_cell_score))
```

```r
# --- Expand to long format (one row per patient per time point) ---
# Each patient contributes 3 rows (one per time point)

longitudinal_data <- patient_data %>%
  # crossing() creates all combinations of patient × time (Cartesian product)
  tidyr::crossing(time_months = time_points) %>%
  mutate(
    # eGFR at each time point:
    # baseline + (individual decline rate × time) + measurement noise
    # The noise term (rnorm) represents lab measurement error (~2 mL/min SD)
    eGFR = eGFR_baseline + decline_rate * (time_months / 6) +
           rnorm(n(), mean = 0, sd = 2),

    # Floor eGFR at 10 (dialysis threshold - patients are removed from study)
    eGFR = pmax(10, eGFR),

    # Time as factor for some analyses (categorical: "Baseline", "6mo", "12mo")
    time_factor = factor(time_months,
                         levels = c(0, 6, 12),
                         labels = c("Baseline", "6 months", "12 months"))
  ) %>%
  # Convert condition and sex to factors (important for regression contrast coding)
  mutate(
    condition = factor(condition, levels = c("Healthy", "CKD")),  # Healthy is reference
    sex = factor(sex, levels = c("Female", "Male"))               # Female is reference
  )

cat("Longitudinal dataset created:\n")
```

```
## Longitudinal dataset created:
```

```r
cat("  Rows:", nrow(longitudinal_data), "(should be", n_total * 3, ")\n")
```

```
##   Rows: 360 (should be 360 )
```

```r
cat("  Patients:", n_distinct(longitudinal_data$patient_id), "\n")
```

```
##   Patients: 120
```

```r
cat("  Columns:", paste(names(longitudinal_data), collapse = ", "), "\n\n")
```

```
##   Columns: patient_id, condition, age, sex, eGFR_baseline, monocyte_score, t_cell_score, decline_rat
```

```r
# ============================================================================
# SECTION 2: DESCRIPTIVE STATISTICS (TABLE 1)
# ============================================================================
# Every clinical paper has a "Table 1" - baseline characteristics of the cohort; showing:
#   1. Continuous variables: mean ± SD (or median [IQR] if non-normal)
#   2. Categorical variables: n (%)
#   3. P-value for group differences (t-test for continuous, chi-squared for categorical)
#
# WHY TABLE 1 MATTERS:
#   If CKD patients are much older than controls, and eGFR naturally declines with age,
#   we need to adjust for age in the models. Table 1 reveals potential confounders.

cat("=== TABLE 1: BASELINE CHARACTERISTICS ===\n\n")
```

```
## === TABLE 1: BASELINE CHARACTERISTICS ===
```

```
# Use only baseline time point for Table 1
baseline_data <- longitudinal_data %>%
  filter(time_months == 0)

# --- Continuous variables: mean ± SD by condition ---
# summarise() aggregates data as needed
table1_continuous <- baseline_data %>%
  group_by(condition) %>%
  summarise(
    n            = n(),
    Age_mean     = round(mean(age), 1),
    Age_sd       = round(sd(age), 1),
    eGFR_mean    = round(mean(eGFR), 1),
    eGFR_sd      = round(sd(eGFR), 1),
    Mono_mean    = round(mean(monocyte_score), 3),
    Mono_sd      = round(sd(monocyte_score), 3),
    Tcell_mean   = round(mean(t_cell_score), 3),
    Tcell_sd     = round(sd(t_cell_score), 3),
    .groups = "drop"
  )

cat("Continuous Variables (Mean ± SD):\n")
```

```
## Continuous Variables (Mean ± SD):
```

```
print(table1_continuous)
```

```
## # A tibble: 2 x 10
##   condition     n Age_mean Age_sd eGFR_mean eGFR_sd Mono_mean Mono_sd Tcell_mean Tcell_sd
##   <fct>     <int>    <dbl>  <dbl>     <dbl>   <dbl>     <dbl>   <dbl>      <dbl>    <dbl>
## 1 Healthy      40     48.8   14.6      89      11.6      0.302   0.119      0.557    0.141
## 2 CKD          80     62.2   12.9      51.5    11.4      0.641   0.158      0.441    0.152
```

```
# --- Sex distribution by condition ---
table1_sex <- baseline_data %>%
  count(condition, sex) %>%
  group_by(condition) %>%
  mutate(pct = round(100 * n / sum(n), 1)) %>%
  ungroup()

cat("\nSex Distribution:\n")
```

```
##
## Sex Distribution:
```

```
print(table1_sex)
```

```
## # A tibble: 4 x 4
##   condition sex        n   pct
##   <fct>     <fct>  <int> <dbl>
## 1 Healthy   Female    23  57.5
## 2 Healthy   Male      17  42.5
## 3 CKD       Female    24  30
## 4 CKD       Male      56  70
```

```r
# --- Statistical tests for group differences ---
# t.test() for continuous variables (tests if means differ between CKD vs Healthy)
# Use var.equal = FALSE (Welch's t-test) - when group sizes/variances differ

ckd_base     <- baseline_data %>% filter(condition == "CKD")
healthy_base <- baseline_data %>% filter(condition == "Healthy")

age_test  <- t.test(ckd_base$age, healthy_base$age, var.equal = FALSE)
egfr_test <- t.test(ckd_base$eGFR, healthy_base$eGFR, var.equal = FALSE)
mono_test <- t.test(ckd_base$monocyte_score, healthy_base$monocyte_score, var.equal = FALSE)

# chisq.test() for categorical variables
sex_table   <- table(baseline_data$condition, baseline_data$sex)
sex_test    <- chisq.test(sex_table)

cat("\nGroup Comparison P-values:\n")
```

```
##
## Group Comparison P-values:
```

```r
cat(sprintf("  Age:            p = %.4f\n", age_test$p.value))
```

```
##   Age:            p = 0.0000
```

```r
cat(sprintf("  Baseline eGFR:  p = %.4f\n", egfr_test$p.value))
```

```
##   Baseline eGFR:  p = 0.0000
```

```r
cat(sprintf("  Monocyte score: p = %.4f\n", mono_test$p.value))
```

```
##   Monocyte score: p = 0.0000
```

```r
cat(sprintf("  Sex (chi-sq):   p = %.4f\n", sex_test$p.value))
```

```
##   Sex (chi-sq):   p = 0.0067
```

```r
# INTERPRETATION GUIDE:
# - eGFR p-value should be highly significant. By design, the simulated CKD cohort
#   has a lower baseline kidney function.
# - Monocyte score p-value should be significant (elevated in CKD by design)
# - Age p-value should be significant (we intentionally simulated older CKD
#   patients to reflect real-world epidemiological data.)
# - Sex p-value would be significant because we intentionally simulated a higher
#   proportion of males in the CKD group to reflect real-world epidemiological data.


# ============================================================================
# SECTION 3: VISUALISATION
# ============================================================================
# Visualise the data BEFORE running statistical models.
# Plots reveal: outliers, non-linearity, missing data patterns, batch effects.
#
# ggplot2 GRAMMAR REMINDER:
#   ggplot(data, aes(x=..., y=..., color=...)) +    # map variables to aesthetics
#   geom_point() +                                  # choose a geometry
#   facet_wrap(~ variable) +                        # panel by variable
#   theme_minimal() +                               # clean theme
```

```r
#   labs(title=..., x=..., y=...)                    # labels

cat("\n=== SECTION 3: VISUALISATION ===\n")
```

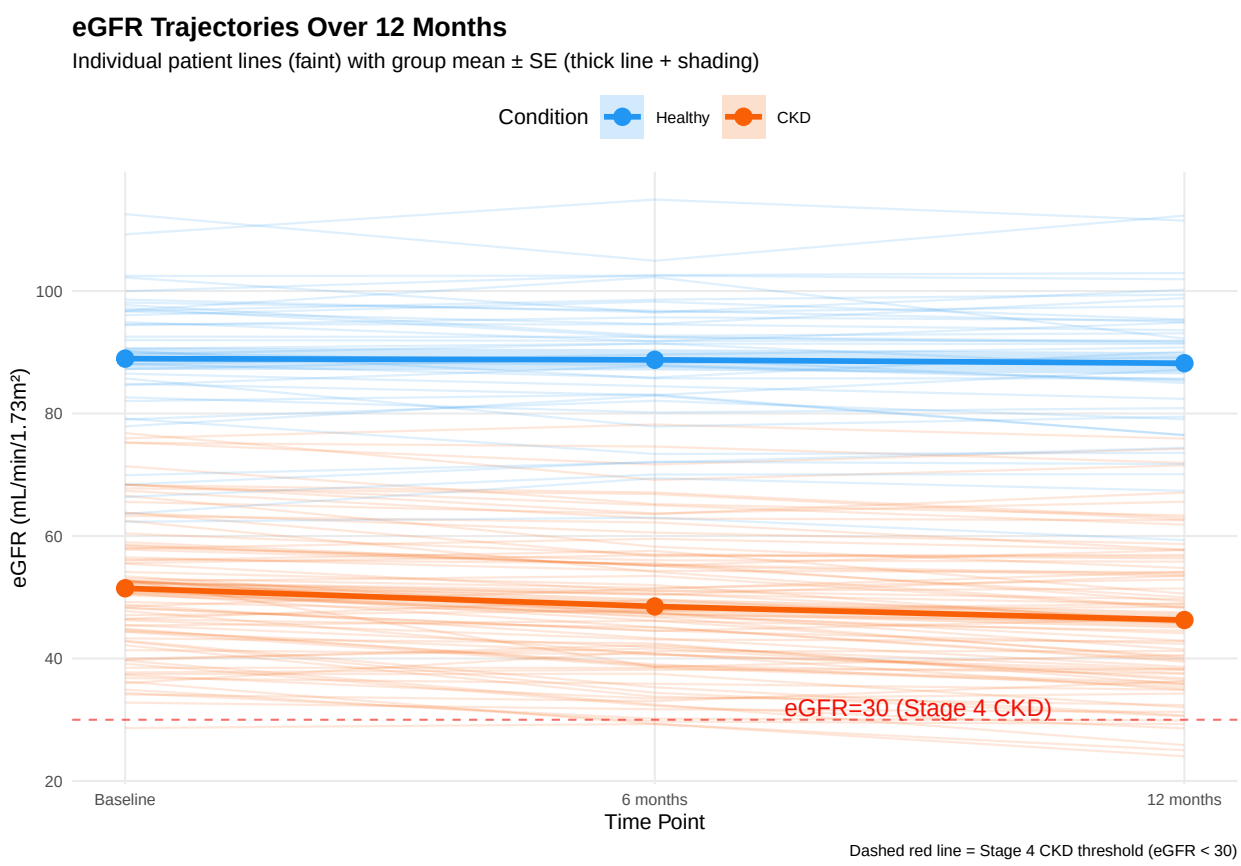
```
##
## === SECTION 3: VISUALISATION ===
```

```r
# --- PLOT 1: eGFR Trajectories Over Time ---
# Goal: Show individual patient trajectories + group mean trend
# Spaghetti plot - each line = one patient
# CKD patients decline while Healthy stay stable

# Define colour palette - using CKD-appropriate colours
condition_colors <- c("Healthy" = "#2196F3",   # Blue for healthy
                      "CKD"     = "#F96006")    # Red for disease

# Individual trajectories (thin, semi-transparent lines)
p_egfr_traj <- ggplot(longitudinal_data,
                      aes(x = time_months, y = eGFR,
                          group = patient_id, color = condition)) +

  # Individual lines
  geom_line(alpha = 0.15, linewidth = 0.4) +

  # Group mean lines
  # stat_summary computes mean at each time point and draws a line
  stat_summary(aes(group = condition),
               fun = mean, geom = "line",
               linewidth = 1, linetype = "solid") +

  # Add mean points at each time point
  stat_summary(aes(group = condition),
               fun = mean, geom = "point",
               size = 3, shape = 16) +

  # Add shaded confidence band around group mean (±1 SE)
  stat_summary(aes(group = condition, fill = condition),
               fun.data = mean_se, geom = "ribbon",
               alpha = 0.2, color = NA) +

  # Reference line: eGFR = 30 is the threshold for Stage 4 CKD (high risk)
  geom_hline(yintercept = 30, linetype = "dashed", color = "#F01006", alpha = 0.6) +
  annotate("text", x = 10.5, y = 32, label = "eGFR=30 (Stage 4 CKD)",
           size = 3, color = "#F01006", hjust = 1) +

  scale_color_manual(values = condition_colors) +
  scale_fill_manual(values = condition_colors) +
  scale_x_continuous(breaks = c(0, 6, 12),
                     labels = c("Baseline", "6 months", "12 months")) +

  labs(
    title    = "eGFR Trajectories Over 12 Months",
    subtitle = "Individual patient lines (faint) with group mean ± SE (thick line + shading)",
    x        = "Time Point",
```

```
    y        = "eGFR (mL/min/1.73m²)",
    color    = "Condition",
    fill     = "Condition",
    caption  = "Dashed red line = Stage 4 CKD threshold (eGFR < 30)"
  ) +
  theme_minimal(base_size = 8) +
  theme(
    plot.title    = element_text(face = "bold"),
    legend.position = "top",
    panel.grid.minor = element_blank()
  )

ggsave(file.path(plot_dir, "01_eGFR_trajectories.png"),
       p_egfr_traj, width = 8, height = 6, dpi = 300)
cat("Saved: 01_eGFR_trajectories.png\n")
```

```
## Saved: 01_eGFR_trajectories.png
```

```
print(p_egfr_traj)
```



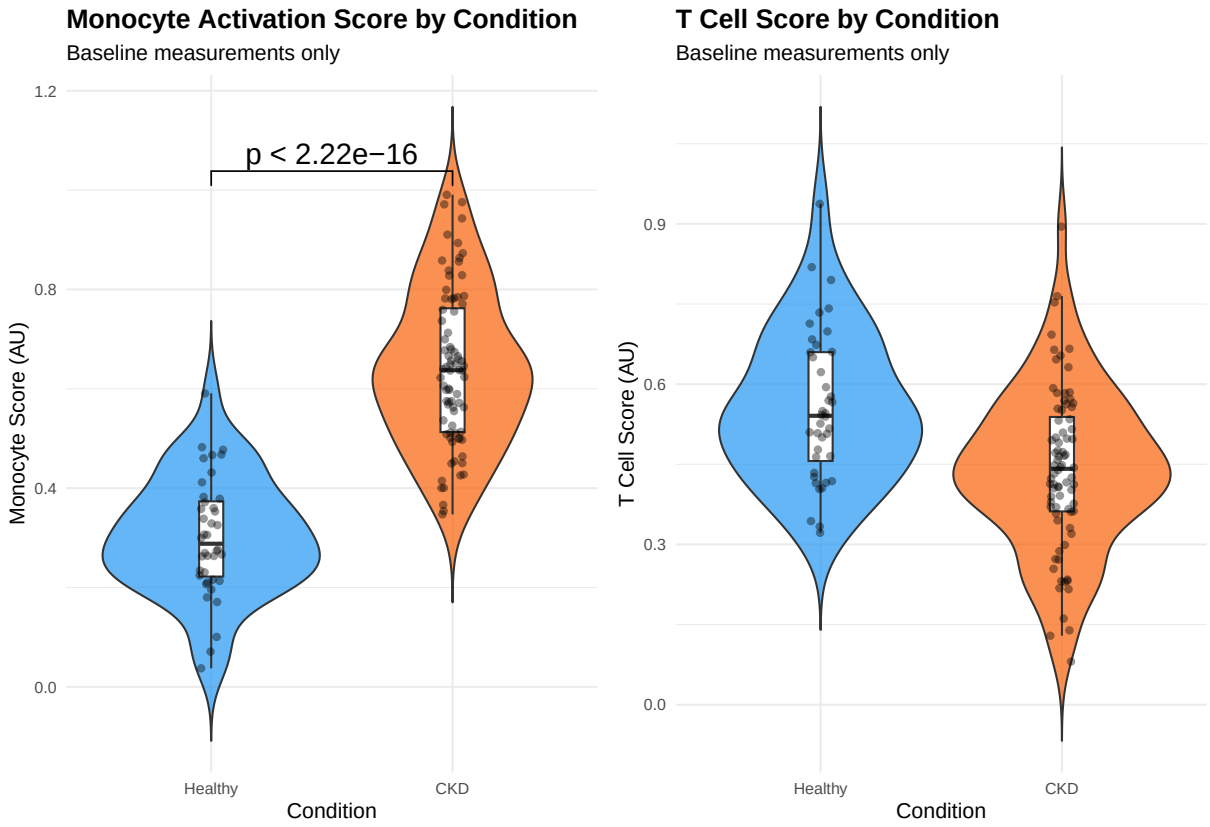

```
# --- PLOT 2: Immune Score Distributions by Condition ---
# Violin plot - shows the full distribution shape
# Useful for spotting bimodality (two subpopulations within CKD)

# Define exactly which 'condition's to compare (required for adding the automated significance)
comparison <- list( c("Healthy", "CKD") )
```

```r
p_mono_violin <- ggplot(baseline_data,
                        aes(x = condition, y = monocyte_score, fill = condition)) +

  # Violin: shows the distribution density
  geom_violin(trim = FALSE, alpha = 0.7) +

  # Boxplot inside the violin - shows median and IQR
  # width=0.1 makes it narrow so it sits inside the violin
  geom_boxplot(width = 0.1, fill = "white", outlier.shape = NA) +

  # Individual points: jittered to avoid overplotting
  geom_jitter(width = 0.05, size = 1, alpha = 0.4) +

  scale_fill_manual(values = condition_colors) +

  # Add the automated statistics
  stat_compare_means(
    comparisons = comparison,
    method = "t.test",    # Or "wilcox.test" for non-parametric
    label = "p.format"    # Or "p.signif" for *** stars)
  ) +

  labs(
    title    = "Monocyte Activation Score by Condition",
    subtitle = "Baseline measurements only",
    x        = "Condition",
    y        = "Monocyte Score (AU)",
    fill     = "Condition"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"),
        legend.position = "none")

p_tcell_violin <- ggplot(baseline_data,
                         aes(x = condition, y = t_cell_score, fill = condition)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white", outlier.shape = NA) +
  geom_jitter(width = 0.05, size = 1, alpha = 0.4) +
  scale_fill_manual(values = condition_colors) +
  labs(
    title    = "T Cell Score by Condition",
    subtitle = "Baseline measurements only",
    x        = "Condition",
    y        = "T Cell Score (AU)",
    fill     = "Condition"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"),
        legend.position = "none")

# Combine using patchwork
p_immune_combined <- p_mono_violin | p_tcell_violin
ggsave(file.path(plot_dir, "02_immune_scores_violin.png"),
```

```
       p_immune_combined, width = 10, height = 6, dpi = 300)
cat("Saved: 02_immune_scores_violin.png\n")
```

```
## Saved: 02_immune_scores_violin.png
```

```
print(p_immune_combined)
```




```
# --- PLOT 3: Scatter - Monocyte Score vs eGFR change ---
# Does higher monocyte activation at baseline predict more eGFR decline?
# This is the core biological hypothesis visualised directly.

# Calculate eGFR change from baseline to 12 months for each CKD patient
egfr_change <- longitudinal_data %>%
  filter(condition == "CKD") %>%
  select(patient_id, time_months, eGFR, monocyte_score) %>%
  pivot_wider(names_from = time_months, values_from = eGFR,
              names_prefix = "eGFR_t") %>%
  mutate(eGFR_change = `eGFR_t12` - `eGFR_t0`)  # negative = decline

p_scatter <- ggplot(egfr_change,
                    aes(x = monocyte_score, y = eGFR_change)) +
  geom_point(alpha = 0.6, color = "#F96006", size = 2) +
  geom_smooth(method = "lm", se = TRUE, color = "#F01006", fill = "pink") +

  # Automatically calculate and plot the correlation (r) AND its significance (p)
  stat_cor(method = "pearson", label.x.npc = "left", label.y.npc = "bottom") +

  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
```

```
  labs(
    title    = "Monocyte Score vs. eGFR Change (CKD patients only)",
    subtitle = "Negative y-axis = eGFR decline over 12 months",
    x        = "Baseline Monocyte Score (AU)",
    y        = "eGFR Change at 12 months (mL/min/1.73m²)"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"))

ggsave(file.path(plot_dir, "03_monocyte_vs_egfr_change.png"),
       p_scatter, width = 7, height = 6, dpi = 300)
```

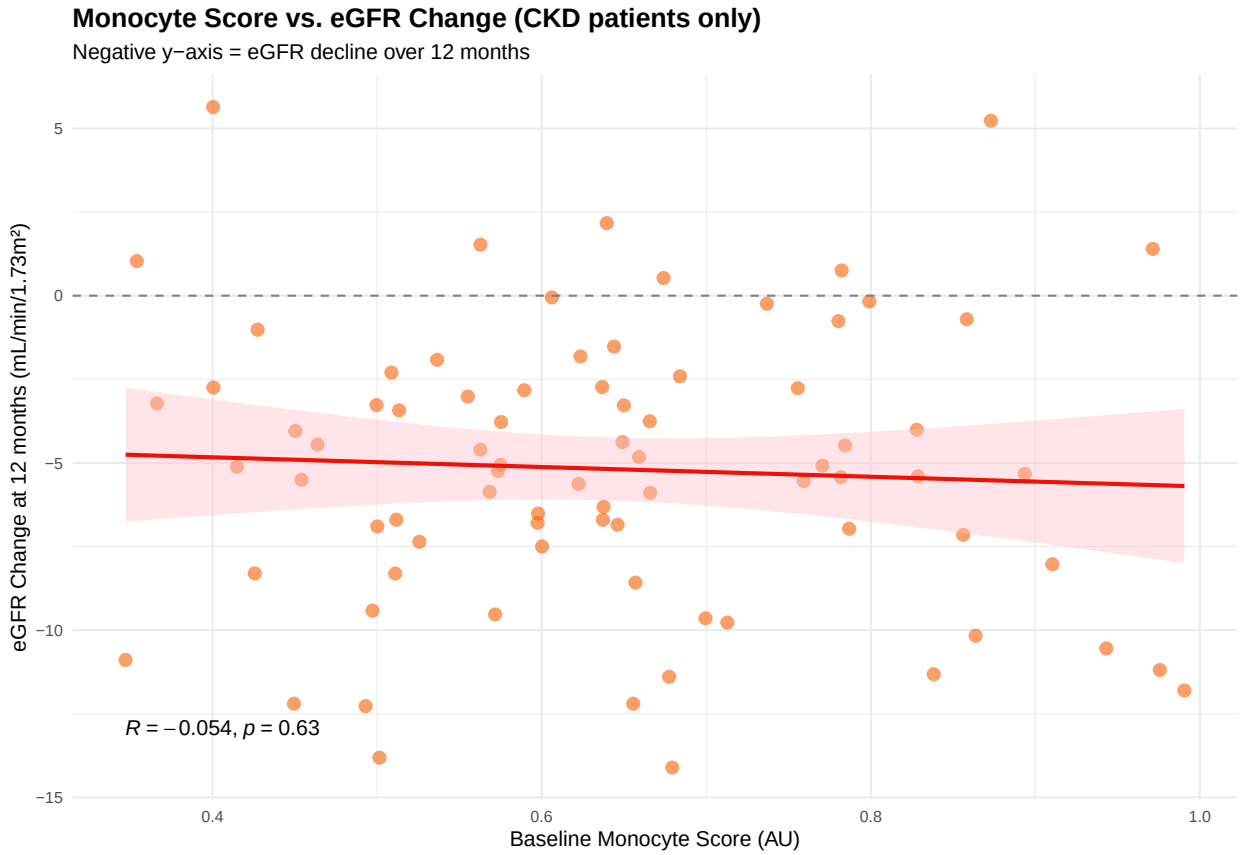
```
## `geom_smooth()` using formula = 'y ~ x'
```

```
cat("Saved: 03_monocyte_vs_egfr_change.png\n")
```

```
## Saved: 03_monocyte_vs_egfr_change.png
```

```
print(p_scatter)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
# ============================================================================
# SECTION 4: MIXED EFFECTS MODELS (lme4)
# ============================================================================
#
# THE FUNDAMENTAL PROBLEM WITH REPEATED MEASURES:
```

```r
# Standard linear regression assumes observations are INDEPENDENT.
# But in longitudinal data, three measurements from the same patient are NOT independent.
# Patient PT001 at 0, 6, and 12 months share the same person and therefore same genetics,
# same lifestyle, same disease severity. These correlations violate regression assumptions.
#
# If we use plain lm() ignoring this, we will:
#   1. Underestimate standard errors (→ false positives)
#   2. Not correctly separate "within-patient" from "between-patient" effects
#
# MIXED EFFECTS MODELS solve this by:
#   - FIXED EFFECTS: population-average effects (what we care about: does CKD affect eGFR?)
#   - RANDOM EFFECTS: patient-specific deviations (each patient has their own "offset")
#     The random effects absorb the correlation within patients
#
# MODEL NOTATION: lmer(eGFR ~ time * condition + (1|patient_id), data=...)
#   - eGFR: outcome variable
#   - time * condition: fixed effects (main effects + interaction)
#     The interaction tests: "does the effect of TIME differ by CONDITION?"
#     i.e., "do CKD patients decline faster than Healthy?"
#   - (1|patient_id): random intercept per patient
#     Each patient is allowed their own baseline eGFR level
#     This is the minimal random effects structure for repeated measures
#
# WHY NOT ALSO (time|patient_id)?
#   A random slope for time would additionally allow each patient their own
#   rate of change. This is more flexible but requires more data to estimate.
#   For 3 time points, a random intercept is usually sufficient.

cat("\n=== SECTION 4: MIXED EFFECTS MODELS ===\n\n")
```

```
##
## === SECTION 4: MIXED EFFECTS MODELS ===
```

```r
# --- MODEL 1: Does CKD condition affect eGFR trajectory? ---
# This is confirmatory: we know CKD patients have lower eGFR.
# But this tests whether the RATE OF CHANGE differs between groups.
#
# Fixed effects interpretation:
#   Intercept: mean eGFR in Healthy group at time=0 (baseline)
#   time_months: slope of eGFR vs time in Healthy group (should be ~0)
#   conditionCKD: how much lower eGFR is in CKD vs Healthy at baseline
#   time_months:conditionCKD: INTERACTION = extra slope in CKD vs Healthy
#     A negative value means CKD declines faster over time

cat("--- Model 1: eGFR ~ time × condition ---\n")
```

```
## --- Model 1: eGFR ~ time × condition ---
```

```r
model1 <- lmer(
  eGFR ~ time_months * condition + (1 | patient_id),
  data = longitudinal_data,
  REML = TRUE  # REML = Restricted Maximum Likelihood
               # Use REML for estimating random effects (less biased than ML)
               # Use ML (REML=FALSE) when comparing models with different fixed effects
)
```

```r
cat("\nModel 1 Summary:\n")
```

```
##
## Model 1 Summary:
```

```r
print(summary(model1))
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method ['lmerModLmerTest']
## Formula: eGFR ~ time_months * condition + (1 | patient_id)
##    Data: longitudinal_data
##
## REML criterion at convergence: 2163.7
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.55523 -0.59399 -0.02542  0.54223  2.53763
##
## Random effects:
##  Groups     Name        Variance Std.Dev.
##  patient_id (Intercept) 127.815  11.31
##  Residual                 5.907   2.43
## Number of obs: 360, groups:  patient_id, 120
##
## Fixed effects:
##                            Estimate Std. Error        df t value Pr(>|t|)
## (Intercept)                89.02532    1.82166 123.39985  48.871  < 2e-16 ***
## time_months                -0.06283    0.04529 238.00000  -1.387    0.167
## conditionCKD              -37.68601    2.23106 123.39986 -16.891  < 2e-16 ***
## time_months:conditionCKD  -0.36906    0.05547 238.00000  -6.654 1.95e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) tm_mnt cndCKD
## time_months -0.149
## conditinCKD -0.816  0.122
## tm_mnth:CKD  0.122 -0.816 -0.149
```

```r
patient_var  <- as.data.frame(VarCorr(model1))$vcov[1]
residual_var <- as.data.frame(VarCorr(model1))$vcov[2]
icc <- patient_var / (patient_var + residual_var)
cat(sprintf("\nModel 1 - Intraclass Correlation (ICC): %.3f\n", icc))
```

```
##
## Model 1 - Intraclass Correlation (ICC): 0.956
```

```r
cat(sprintf("Interpretation: %.1f%% of eGFR variance is between-patient (vs within-patient noise)\n",
    icc * 100))
```

```
## Interpretation: 95.6% of eGFR variance is between-patient (vs within-patient noise)
```

```r
cat("ICC > 0.5 = substantial clustering => mixed model is essential!\n\n")
```

```
## ICC > 0.5 = substantial clustering => mixed model is essential!
```

```r
# --- Understanding the model output ---
# Random effects section:
```

```
#   Groups: patient_id - this tells us the variance between patients
#   Residual: variance within patients (unexplained noise)
#   ICC (Intraclass Correlation) = patient variance / (patient + residual variance)
#   ICC tells us: "what fraction of total variance is due to patient identity?"
#   High ICC (>0.5) → patients are very different from each other


# Extract fixed effects with confidence intervals
# confint() on lmer uses parametric bootstrap or profile likelihood
# Use broom.mixed::tidy() for a clean data frame
model1_tidy <- tidy(model1, effects = "fixed", conf.int = TRUE)
cat("Fixed Effects with 95% CI:\n")
```

```
## Fixed Effects with 95% CI:
```

```
print(model1_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

```
## # A tibble: 4 x 5
##   term                      estimate conf.low conf.high  p.value
##   <chr>                        <dbl>    <dbl>     <dbl>    <dbl>
## 1 (Intercept)               89.0      85.4     92.6    1.32e-82
## 2 time_months               -0.0628   -0.152    0.0264 1.67e- 1
## 3 conditionCKD             -37.7     -42.1    -33.3    6.93e-34
## 4 time_months:conditionCKD  -0.369    -0.478   -0.260  1.95e-10
```

```
# --- MODEL 2: Does monocyte score predict eGFR decline? ---
# The KEY CLINICAL QUESTION: does baseline immune phenotype predict future kidney function?
#
# We include monocyte_score * time_months interaction because:
#   We hypothesise that higher monocyte score → faster eGFR decline
#   = the effect of monocyte score on eGFR CHANGES over time
#   This is a "dynamic predictor" analysis.
#
# We control for age and sex as potential confounders.
# We analyse CKD patients only for this model (no healthy controls)
#   because the hypothesis is specifically about disease progression

cat("\n--- Model 2: eGFR ~ monocyte_score × time + age + sex (CKD only) ---\n")
```

```
##
## --- Model 2: eGFR ~ monocyte_score × time + age + sex (CKD only) ---
```

```
ckd_longitudinal <- longitudinal_data %>%
  filter(condition == "CKD")

model2 <- lmer(
  eGFR ~ monocyte_score * time_months + age + sex + (1 | patient_id),
  data = ckd_longitudinal,
  REML = TRUE
)

cat("\nModel 2 Summary:\n")
```

```
##
## Model 2 Summary:
```

```r
print(summary(model2))
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method ['lmerModLmerTest']
## Formula: eGFR ~ monocyte_score * time_months + age + sex + (1 | patient_id)
##    Data: ckd_longitudinal
##
## REML criterion at convergence: 1444.8
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.54989 -0.53709 -0.05733  0.53324  2.51715
##
## Random effects:
##  Groups     Name        Variance Std.Dev.
##  patient_id (Intercept) 133.663  11.561
##  Residual                6.215   2.493
## Number of obs: 240, groups:  patient_id, 80
##
## Fixed effects:
##                             Estimate Std. Error        df t value Pr(>|t|)
## (Intercept)                 53.85434    8.53701  77.45543   6.308 1.61e-08 ***
## monocyte_score              -3.01393    8.41939  79.49413  -0.358   0.7213
## time_months                 -0.35423    0.13840 158.00000  -2.559   0.0114 *
## age                          0.01907    0.10184  75.99999   0.187   0.8520
## sexMale                     -2.52826    2.84377  76.00000  -0.889   0.3768
## monocyte_score:time_months  -0.12116    0.20977 158.00000  -0.578   0.5644
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) mncyt_ tm_mnt age    sexMal
## monocyt_scr -0.615
## time_months -0.097  0.145
## age         -0.718 -0.031  0.000
## sexMale     -0.235  0.025  0.000 -0.019
## mncyt_scr:_  0.094 -0.149 -0.971  0.000  0.000
```

```r
# Extract and display key results
model2_tidy <- tidy(model2, effects = "fixed", conf.int = TRUE)
cat("\nFixed Effects with 95% CI:\n")
```

```
##
## Fixed Effects with 95% CI:
```

```r
print(model2_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

```
## # A tibble: 6 x 5
##   term                        estimate conf.low conf.high      p.value
##   <chr>                          <dbl>    <dbl>     <dbl>        <dbl>
## 1 (Intercept)                   53.9      36.9     70.9    0.0000000161
## 2 monocyte_score                -3.01    -19.8     13.7    0.721
## 3 time_months                   -0.354    -0.628   -0.0809 0.0114
## 4 age                            0.0191   -0.184    0.222  0.852
## 5 sexMale                       -2.53     -8.19     3.14   0.377
## 6 monocyte_score:time_months    -0.121    -0.535    0.293  0.564
```

```
# --- PLOT 4: Coefficient Plot (Forest Plot) for Model 2 ---
# A forest plot displays effect sizes with confidence intervals.
# The standard way to present regression results in clinical papers.
# Horizontal line at 0: if CI crosses zero, effect is not significant.

# Filter to key predictors (exclude intercept for visual clarity)
coef_plot_data <- model2_tidy %>%
  filter(effect == "fixed", term != "(Intercept)") %>%
  mutate(
    term = recode(term,
                  "monocyte_score"            = "Monocyte Score (baseline)",
                  "time_months"               = "Time (per month)",
                  "age"                       = "Age (per year)",
                  "sexMale"                   = "Sex: Male",
                  "monocyte_score:time_months" = "Monocyte × Time (interaction)"),
    significant = p.value < 0.05
  )

p_forest_lmer <- ggplot(coef_plot_data,
                        aes(x = estimate, y = reorder(term, estimate),
                            color = significant)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "#757575") +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.2, linewidth = 1) +
  geom_point(size = 3) +
  scale_color_manual(values = c("TRUE" = "#F96006", "FALSE" = "#757575"),
                     labels = c("TRUE" = "p < 0.05", "FALSE" = "p >= 0.05")) +
  labs(
    title    = "Model 2: Fixed Effect Estimates",
    subtitle = "eGFR ~ monocyte_score × time + age + sex (CKD patients only)",
    x        = "Estimate (mL/min/1.73m² change per unit predictor)",
    y        = "",
    color    = "Significance"
  ) +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold"))

ggsave(file.path(plot_dir, "04_lmer_coefficients.png"),
       p_forest_lmer, width = 9, height = 5, dpi = 300)
```

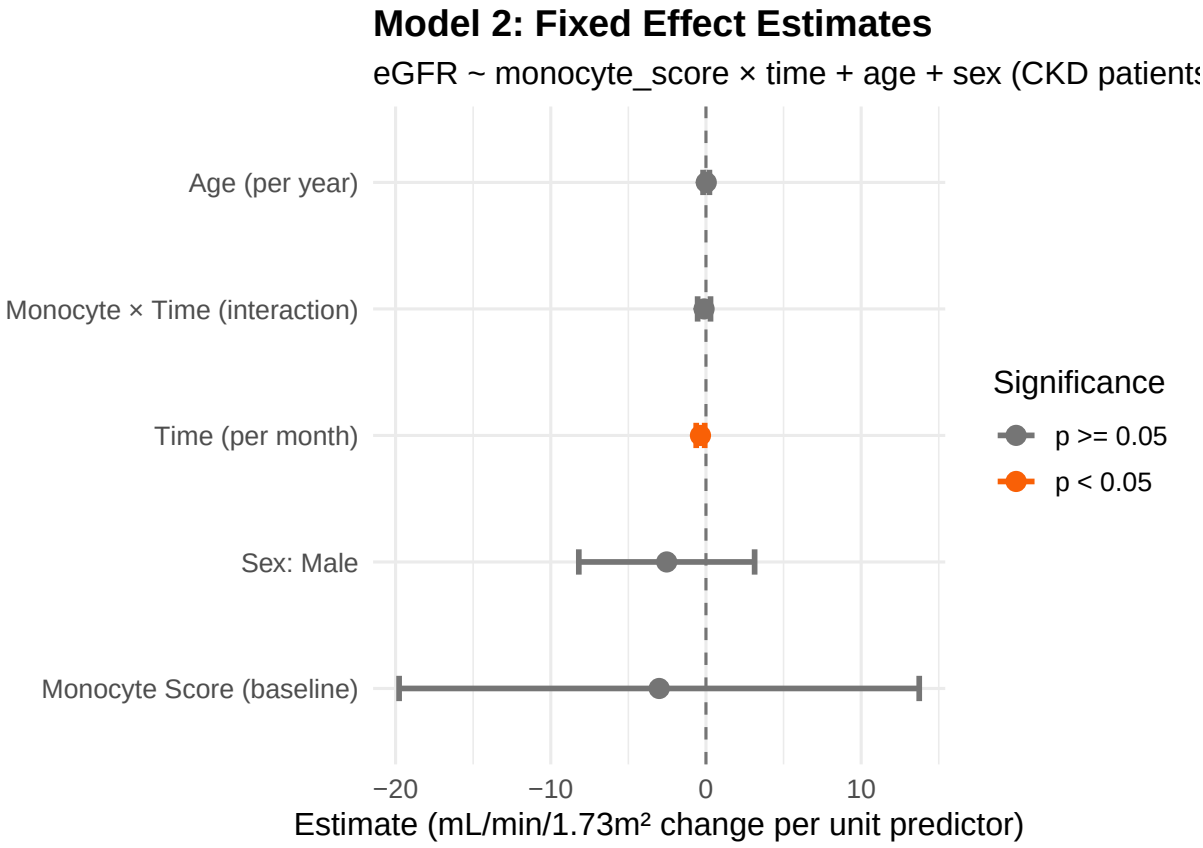
```
## `height` was translated to `width`.
```

```
cat("\nSaved: 04_lmer_coefficients.png\n")
```

```
##
## Saved: 04_lmer_coefficients.png
```

```
print(p_forest_lmer)
```

```
## `height` was translated to `width`.
```




```
# --- INTERPRETING THE INTERACTION TERM ---
cat("\n--- KEY RESULT INTERPRETATION ---\n")
```

```
##
## --- KEY RESULT INTERPRETATION ---
```

```
interaction_term <- model2_tidy %>%
  filter(term == "monocyte_score:time_months")

if (nrow(interaction_term) > 0) {
  est <- interaction_term$estimate
  p   <- interaction_term$p.value
  cat(sprintf("Monocyte × Time interaction:   = %.3f, p = %.4f\n", est, p))
  cat("This means: for every 1-unit increase in monocyte score,\n")
  cat(sprintf("  eGFR declines an additional %.3f mL/min per MONTH\n", est))
  cat(sprintf("  = %.3f mL/min per 12 months (one year)\n", est * 12))
  if (p < 0.05) {
    cat("  SIGNIFICANT: Higher monocyte activation predicts faster kidney function decline\n")
  } else {
    cat("  Not significant at p < 0.05 (may need larger sample)\n")
  }
}
```

```
## Monocyte × Time interaction:   = -0.121, p = 0.5644
## This means: for every 1-unit increase in monocyte score,
##   eGFR declines an additional -0.121 mL/min per MONTH
##   = -1.454 mL/min per 12 months (one year)
##   Not significant at p < 0.05 (may need larger sample)
```

```r
# ============================================================================
# SECTION 5: SURVIVAL ANALYSIS
# ============================================================================
#
# WHAT IS SURVIVAL ANALYSIS?
```

```r
# Survival analysis answers the question: "How long until an event occurs?"
# The "event" doesn't have to be death - it can be:
#   - Time to kidney failure (eGFR drops below 30)
#   - Time to first hospitalisation
#   - Time to treatment response
#
# WHY NOT JUST USE LOGISTIC REGRESSION?
#   Logistic regression: "Did the event happen? Yes/No"
#   Survival analysis: "WHEN did the event happen? And what if we didn't observe it?"
#
# THE CENSORING PROBLEM:
#   A patient who hasn't had kidney failure by the end of the study is "censored".
#   We know they survived AT LEAST until the end of study, but don't know their
#   eventual outcome. We must not:
#     - Exclude them (introduces bias - we'd only analyse sick patients)
#     - Say they had the event (incorrect)
#   Survival analysis handles censoring correctly.
#
# KEY METHODS:
#   1. Kaplan-Meier: Non-parametric estimate of the survival curve
#      No assumptions about the shape of the hazard function
#      !!!Kaplan-Meier completely fails when we introduce continuous variables
#      (like monocyte_score). To plot a KM curve for a continuous variable, we must
#      categorise the data (e.g., chopping scores into "High", "Medium", and "Low").
#      Doing this destroys valuable mathematical nuance. KM cannot
#      adjust for multiple confounding variables at the same time.
#   2. Cox Proportional Hazards: Semi-parametric model
#      h(t x)=h0(t) e^( x) where h0(t) is the baseline hazard [the word "baseline"
#      refers to a person, not a time, the baseline hazard is the hazard rate for a hypothetical
#      patient whose covariate values (x) are all exactly zero], x is the
#      predictor variable (here: monocyte_score - 0.5,
#      and [-0.5] is for mean centering - assuming that 0.5 is a "typical" monocyte score),
#        is the coefficient (here: 2.0).
#      The Individual Component: e^( x) depends only on the individual's
#      characteristics (x) and the coefficient ( ).
#      The Time Component: h0(t) depends only on time (t). This is the baseline hazard,
#      and it can fluctuate wildly as time passes.
#      Therefore, hazard curves for different people must never cross.
#      The fundamental assumption: the hazard for any specific individual
#      is a multiple of the baseline hazard (h0(t)).
#      Asks: does a covariate affect the RATE of the event?
#      "Proportional hazards" assumption: the hazard ratio is constant over time

cat("\n=== SECTION 5: SURVIVAL ANALYSIS ===\n\n")
```

```
##
## === SECTION 5: SURVIVAL ANALYSIS ===
```

```r
# --- Simulate time-to-event data ---
# We define "kidney failure" = eGFR dropping below 30 at any time point
#
# For a more realistic simulation, we generate continuous follow-up times.
# In reality, we'd extract this from EHR (electronic health records).

# Follow-up: up to 36 months (3 years)
# CKD patients with high monocyte scores fail earlier

set.seed(42)

# Generate survival times from an exponential distribution
# Higher monocyte score → shorter survival (parametrised by hazard rate)
# The hazard ( ) in exponential distribution: larger  = events happen sooner

survival_data <- patient_data %>%
  mutate(
    # Base hazard: CKD patients have intrinsic risk; healthy have near-zero risk
    # In a constant-hazard survival model (h(t)= ), the probability of surviving past time t
    # is S(t)=e^(-h t), where e is Euler's number (approximately 2.718),
    # h is the base hazard rate, and t is the time elapsed (in this case, 36 months)
    # The probability of failing within that time is simply 1 minus the survival probability:
    # Failure(t)=1-e^(-h t)

    base_hazard = case_when(
      condition == "CKD"     ~ 0.025,  # ~60% fail within 36 months
      condition == "Healthy" ~ 0.002   # ~7% fail within 36 months (by design)
    ),

    # Monocyte score multiplies the hazard (this is the Cox model assumption)
    # A monocyte score of 0.8 vs 0.3 doubles the hazard in CKD
    individual_hazard = base_hazard * exp(2.0 * (monocyte_score - 0.5)),

    # Draw event time from exponential distribution
    # In survival analysis, the Exponential distribution is unique because it is
    # "memoryless," meaning the risk of the event happening remains constant over time.

    # rexp(n, rate) gives time-to-event; rate = hazard
    event_time_raw = rexp(n(), rate = individual_hazard),

    # Administrative censoring at 36 months (study ends)
    censoring_time = 36,

    # Some patients drop out early (loss to follow-up) - random censoring
    dropout_time = runif(n(), min = 18, max = 60),  # dropout if < 36

    # Observed time = minimum of event time, censoring, dropout
    time_to_event = pmin(event_time_raw, censoring_time, dropout_time),
    time_to_event = round(time_to_event, 1),

    # Event indicator: 1 = kidney failure occurred, 0 = censored
    # A patient is an "event" only if they failed BEFORE being censored
    event = as.integer(event_time_raw <= pmin(censoring_time, dropout_time))
```

```
  )

cat("Survival data summary:\n")
```

```
## Survival data summary:
```

```
cat(sprintf("  CKD events (failures): %d / %d (%.1f%%)\n",
            sum(survival_data$event[survival_data$condition == "CKD"]),
            sum(survival_data$condition == "CKD"),
            100 * mean(survival_data$event[survival_data$condition == "CKD"])))
```

```
##   CKD events (failures): 54 / 80 (67.5%)
```

```
cat(sprintf("  Healthy events: %d / %d (%.1f%%)\n",
            sum(survival_data$event[survival_data$condition == "Healthy"]),
            sum(survival_data$condition == "Healthy"),
            100 * mean(survival_data$event[survival_data$condition == "Healthy"])))
```

```
##   Healthy events: 0 / 40 (0.0%)
```

```
# --- Kaplan-Meier Curves ---
# Surv() creates a survival object: two columns (time, event status)
# survfit() fits the KM estimator
# ggsurvplot() from survminer creates survival plots

km_fit <- survfit(
  Surv(time_to_event, event) ~ condition,
  data = survival_data
)

cat("\nKaplan-Meier Summary:\n")
```

```
##
## Kaplan-Meier Summary:
```

```
print(km_fit)
```

```
## Call: survfit(formula = Surv(time_to_event, event) ~ condition, data = survival_data)
##
##                     n events median 0.95LCL 0.95UCL
## condition=CKD      80     54   18.6    15.3      23
## condition=Healthy 40      0     NA      NA      NA
```

```
# Extract KM summary at specific time points
km_summary <- summary(km_fit, times = c(12, 24, 36))
cat("\nKM Survival Probabilities at 12, 24, 36 months:\n")
```

```
##
## KM Survival Probabilities at 12, 24, 36 months:
```

```
print(data.frame(
  condition = km_summary$strata,
  time      = km_summary$time,
  survival  = round(km_summary$surv, 3),
  lower_CI  = round(km_summary$lower, 3),
  upper_CI  = round(km_summary$upper, 3)
))
```

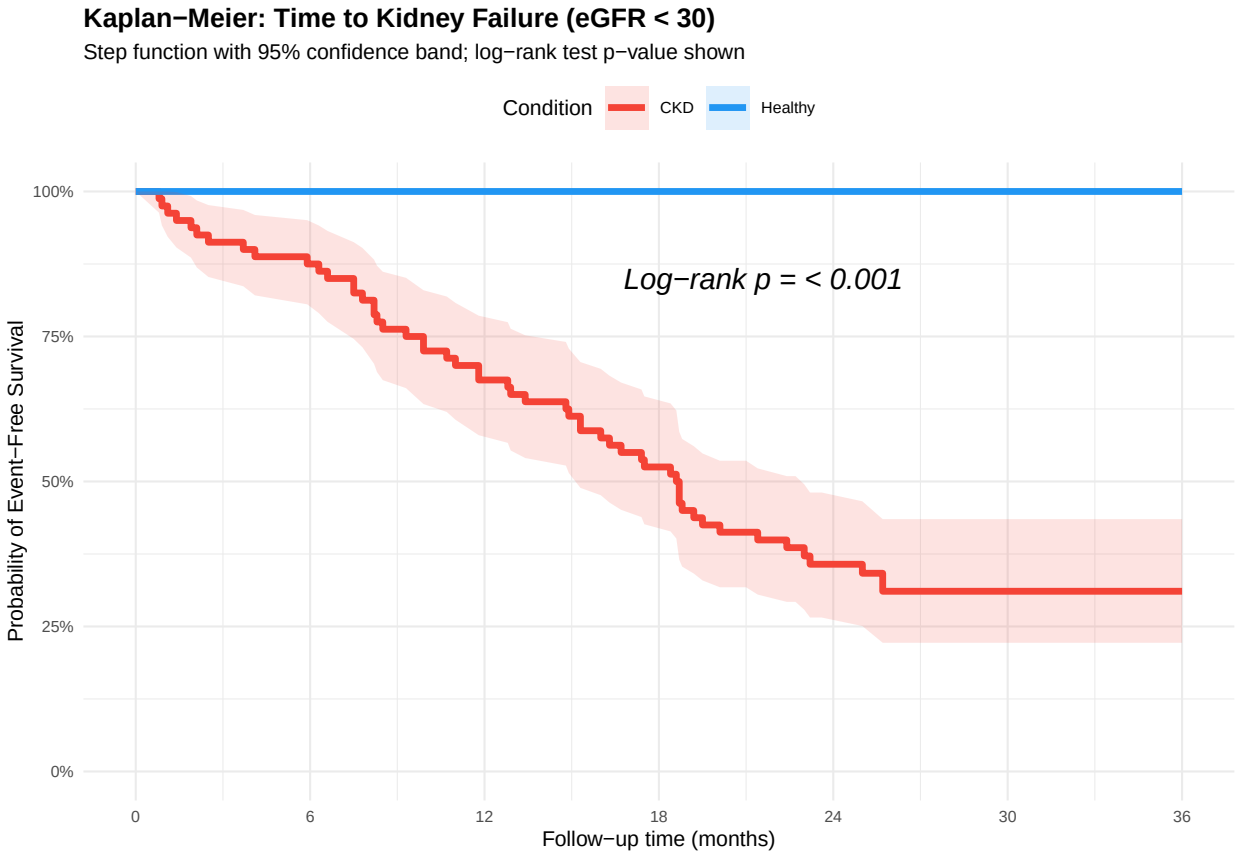
```
##           condition time survival lower_CI upper_CI
## 1     condition=CKD   12    0.675    0.580    0.786
## 2     condition=CKD   24    0.357    0.265    0.481
## 3     condition=CKD   36    0.311    0.222    0.435
## 4 condition=Healthy   12    1.000    1.000    1.000
## 5 condition=Healthy   24    1.000    1.000    1.000
## 6 condition=Healthy   36    1.000    1.000    1.000
```

```r
# --- Build KM plot using ggplot2 directly ---
# We extract the KM curve data from the survfit object and plot it manually.
# This avoids the survminer dependency (survminer is a wrapper around exactly this approach.)
#
# The survfit object contains:
#   $time   = event times
#   $surv   = survival probability at each time
#   $lower, $upper = 95% CI
#   $strata = group labels

km_df <- data.frame(
  time      = km_fit$time,
  surv      = km_fit$surv,
  lower     = km_fit$lower,
  upper     = km_fit$upper,
  strata    = rep(names(km_fit$strata), km_fit$strata)
)

# Clean up strata labels (survfit appends "condition=" prefix)
km_df$condition <- gsub("condition=", "", km_df$strata)

# Add time=0, surv=1 starting points for each group (KM starts at 1)
km_start <- data.frame(
  time      = 0, surv = 1, lower = 1, upper = 1,
  strata    = names(km_fit$strata),
  condition = gsub("condition=", "", names(km_fit$strata))
)
km_df <- rbind(km_start, km_df)

# Log-rank test (tests whether the two KM curves are statistically different)
# The log-rank test is the non-parametric equivalent of a t-test for survival data.
log_rank <- survdiff(Surv(time_to_event, event) ~ condition, data = survival_data)
log_rank_p <- 1 - pchisq(log_rank$chisq, df = length(log_rank$n) - 1)

p_km <- ggplot(km_df, aes(x = time, y = surv, color = condition, fill = condition)) +
  # Step function - KM curves are step functions, not smooth lines
  geom_step(linewidth = 1.2) +
  # 95% confidence band (shaded ribbon)
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.15, color = NA) +

  # Add log-rank p-value annotation
  annotate("text",
           x = max(km_df$time) * 0.6, y = 0.85,
           label = sprintf("Log-rank p = %s",
                           ifelse(log_rank_p < 0.001, "< 0.001",
                                  sprintf("%.3f", log_rank_p))),
```

```
          size = 4, fontface = "italic") +

  scale_color_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
  scale_fill_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  scale_x_continuous(breaks = seq(0, 36, by = 6)) +

  labs(
    title    = "Kaplan-Meier: Time to Kidney Failure (eGFR < 30)",
    subtitle = "Step function with 95% confidence band; log-rank test p-value shown",
    x        = "Follow-up time (months)",
    y        = "Probability of Event-Free Survival",
    color    = "Condition",
    fill     = "Condition"
  ) +
  theme_minimal(base_size = 8) +
  theme(
    plot.title      = element_text(face = "bold"),
    legend.position = "top"
  )

ggsave(file.path(plot_dir, "05_kaplan_meier.png"),
       p_km, width = 8, height = 6, dpi = 300)
cat("\nSaved: 05_kaplan_meier.png\n")
```

```
##
## Saved: 05_kaplan_meier.png
```

```
print(p_km)
```




```r
# --- Build KM plot using ggplot2 directly (alternative - automated) ---
# --- Automated Log-Rank Test ---
# survdiff calculates the test, and broom::glance instantly extracts the p-value
log_rank <- survdiff(Surv(time_to_event, event) ~ condition, data = survival_data)
log_rank_p <- glance(log_rank)$p.value

# --- Automated Kaplan-Meier Plot ---
# ggfortify::autoplot reads the km_fit object directly, builds the data frame,
# injects the t=0 starting points, and generates the step/ribbon layers.
p_km <- autoplot(km_fit,
                 surv.connect = TRUE,     # Adds the t=0, surv=1 starting points
                 conf.int = TRUE,         # Adds the geom_ribbon
                 conf.int.alpha = 0.15,   # Sets ribbon transparency
                 surv.size = 1.2) +       # Sets the line thickness

  # Add log-rank p-value annotation
  annotate("text",
           x = max(km_fit$time) * 0.6, y = 0.85,
           label = sprintf("Log-rank p = %s",
                           ifelse(log_rank_p < 0.001, "< 0.001",
                                  sprintf("%.3f", log_rank_p))),
           size = 4, fontface = "italic") +

  # Set colors and clean up the legend labels.
  # Use the clean names "Healthy" and "CKD" that ggfortify automatically generates
  scale_color_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
  scale_fill_manual(values = c("Healthy" = "#2196F3", "CKD" = "#F44336")) +
```

```
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  scale_x_continuous(breaks = seq(0, 36, by = 6)) +

  # Labels and theming
  labs(
    title    = "Kaplan-Meier: Time to Kidney Failure (eGFR < 30)",
    subtitle = "Automated step function with 95% confidence band; log-rank test p-value shown",
    x        = "Follow-up time (months)",
    y        = "Probability of Event-Free Survival",
    color    = "Condition",
    fill     = "Condition"
  ) +
  theme_minimal(base_size = 8) +
  theme(
    plot.title      = element_text(face = "bold"),
    legend.position = "top"
  )
```

```
## Scale for y is already present.
## Adding another scale for y, which will replace the existing scale.
```

```
# --- Save Plot ---
ggsave(file.path(plot_dir, "05_kaplan_meier_auto.png"),
       p_km, width = 8, height = 6, dpi = 300)
cat("\nSaved: 05_kaplan_meier_auto.png\n")
```

```
##
## Saved: 05_kaplan_meier_auto.png
```

```
# --- Cox Proportional Hazards Model ---
# Cox model: h(t) = h0(t) × exp( x  +  x  + ...)
#   h(t) = hazard at time t (instantaneous rate of event)
#   h0(t) = baseline hazard (the "shape" of the hazard over time - unspecified)
#   exp( ) = HAZARD RATIO (HR)
#     HR > 1: covariate increases risk (faster events)
#     HR < 1: covariate decreases risk (protective)
#     HR = 1: no effect
#
# PROPORTIONAL HAZARDS ASSUMPTION:
#   The ratio of hazards between two groups is CONSTANT over time.
#   e.g., if CKD patients have HR=3 vs healthy, this holds at month 1, 12, 36...
#   We should test this with cox.zph() - a significant test = assumption violated.

cat("\n--- Cox Proportional Hazards Model ---\n")
```

```
##
## --- Cox Proportional Hazards Model ---
```

```
# NOTE: Including `condition` here will produce a warning about infinite coefficients
# because no Healthy patients had an event (by design in our simulation).
# In a real study: either use only CKD patients, or ensure the control group has
# some events. The warning is benign here - the model still estimates the other terms.
cox_model <- coxph(
  Surv(time_to_event, event) ~ monocyte_score + age + sex + condition,
  data = survival_data
```

```
)
```

```
## Warning in coxph.fit(X, Y, istrat, offset, init, control, weights = weights, : Loglik converged befo
## variable 4 ; coefficient may be infinite.
```

```
cat("\nCox Model Summary:\n")
```

```
##
## Cox Model Summary:
```

```
print(summary(cox_model))
```

```
## Call:
## coxph(formula = Surv(time_to_event, event) ~ monocyte_score +
##     age + sex + condition, data = survival_data)
##
##   n= 120, number of events= 54
##
##                         coef  exp(coef)   se(coef)      z Pr(>|z|)
## monocyte_score     1.089e+00  2.971e+00  7.865e-01  1.384    0.166
## age                1.008e-04  1.000e+00  1.083e-02  0.009    0.993
## sexMale            1.768e-01  1.193e+00  3.068e-01  0.576    0.564
## conditionHealthy -1.963e+01  2.983e-09  3.403e+03 -0.006    0.995
##
##                  exp(coef) exp(-coef) lower .95 upper .95
## monocyte_score   2.971e+00  3.366e-01    0.6358    13.878
## age              1.000e+00  9.999e-01    0.9791     1.022
## sexMale          1.193e+00  8.379e-01    0.6541     2.178
## conditionHealthy 2.983e-09  3.352e+08    0.0000       Inf
##
## Concordance= 0.747  (se = 0.032 )
## Likelihood ratio test= 65.34  on 4 df,   p=2e-13
## Wald test            = 2.31  on 4 df,   p=0.7
## Score (logrank) test = 45.75  on 4 df,   p=3e-09
```

```
# Test proportional hazards assumption
# A significant p-value means the HR changes over time → assumption violated
ph_test <- cox.zph(cox_model)
cat("\nProportional Hazards Assumption Test (should be non-significant):\n")
```

```
##
## Proportional Hazards Assumption Test (should be non-significant):
```

```
print(ph_test)
```

```
##                   chisq df    p
## monocyte_score 8.01e-01  1 0.37
## age            5.14e-01  1 0.47
## sex            3.39e-01  1 0.56
## condition      8.38e-09  1 1.00
## GLOBAL         1.59e+00  4 0.81
```

```
# --- Forest Plot of Hazard Ratios ---
# Hazard ratios (HR) are the Cox model equivalent of regression coefficients
# HR is presented on a LOG scale: HR=1 (no effect) maps to 0 on log scale

cox_tidy <- tidy(cox_model, exponentiate = TRUE, conf.int = TRUE)
```

```r
# exponentiate = TRUE converts log(HR) coefficients to HR scale

cat("\nHazard Ratios with 95% CI:\n")
```

```
##
## Hazard Ratios with 95% CI:
```

```r
print(cox_tidy[, c("term", "estimate", "conf.low", "conf.high", "p.value")])
```

```
## # A tibble: 4 x 5
##   term                  estimate conf.low conf.high p.value
##   <chr>                    <dbl>    <dbl>     <dbl>   <dbl>
## 1 monocyte_score   2.97             0.636     13.9    0.166
## 2 age              1.00             0.979      1.02   0.993
## 3 sexMale          1.19             0.654      2.18   0.564
## 4 conditionHealthy 0.00000000298    0        Inf      0.995
```

```r
cox_tidy <- cox_tidy %>%
  # Remove degenerate term: conditionHealthy has HR  0 because no healthy
  # patients had the event (by simulation design). In a real study we would
  # either use CKD as the reference level or analyse CKD patients only.
  filter(term != "conditionHealthy") %>%
  mutate(
    term = recode(term,
                  "monocyte_score" = "Monocyte Score",
                  "age"            = "Age (per year)",
                  "sexMale"        = "Sex: Male",
                  "conditionCKD"   = "Condition: CKD"),
    significant = p.value < 0.05
  )

p_cox_forest <- ggplot(cox_tidy,
                       aes(x = estimate, y = reorder(term, estimate),
                           color = significant)) +
  # Reference line at HR=1 (no effect)
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey50") +
  geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.2, linewidth = 1.2) +
  geom_point(size = 4) +
  # Log scale on x-axis - standard for hazard ratios
  scale_x_log10(
    breaks = c(0.1, 0.25, 0.5, 1, 2, 4, 8, 16),
    labels = c("0.1", "0.25", "0.5", "1", "2", "4", "8", "16")
  ) +
  scale_color_manual(values = c("TRUE" = "#E53935", "FALSE" = "#757575"),
                     labels = c("TRUE" = "p < 0.05", "FALSE" = "p >= 0.05")) +
  labs(
    title    = "Cox Model: Hazard Ratios for Kidney Failure",
    subtitle = "HR > 1 = increased risk; HR < 1 = decreased risk; X-axis is log scale",
    x        = "Hazard Ratio (95% CI)",
    y        = "",
    color    = "Significance"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"),
        panel.grid.minor = element_blank())
```

```
ggsave(file.path(plot_dir, "06_cox_forest_plot.png"),
       p_cox_forest, width = 8, height = 5, dpi = 300)
```

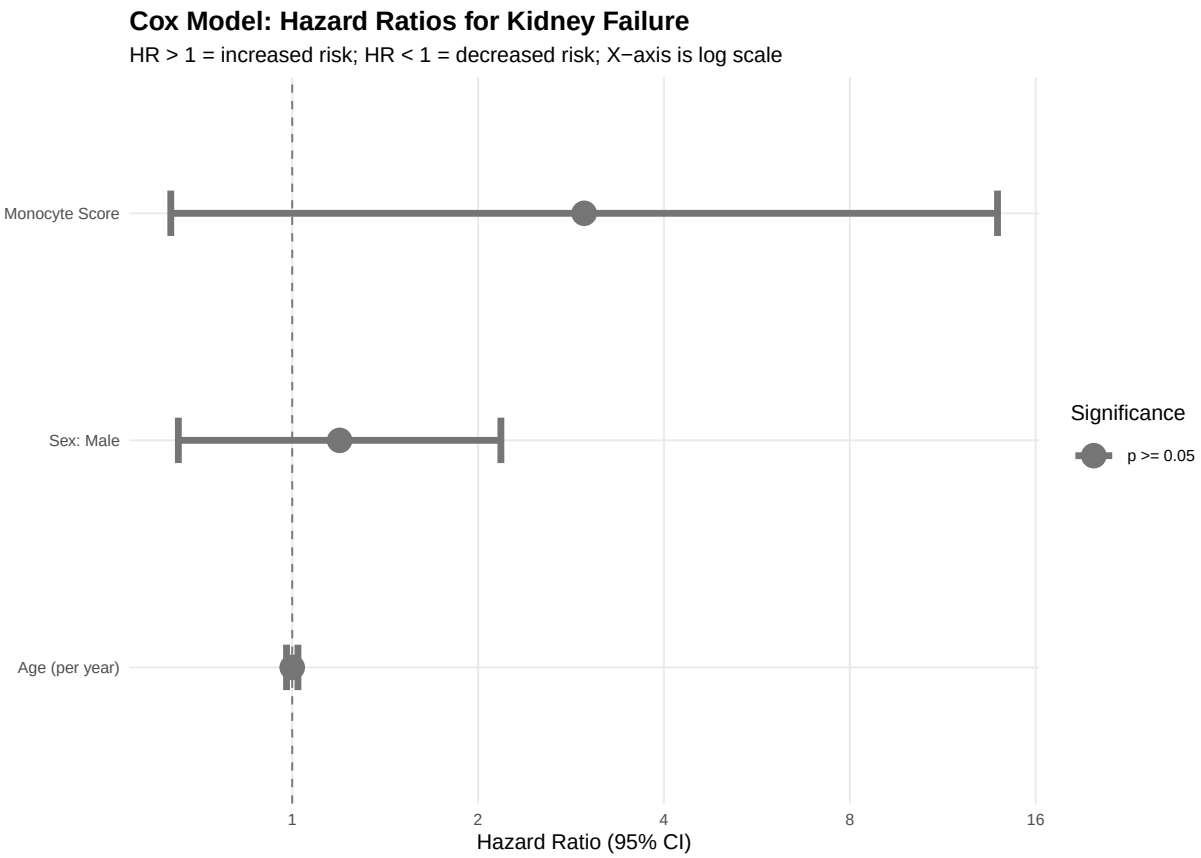
```
## `height` was translated to `width`.
```

```
cat("Saved: 06_cox_forest_plot.png\n")
```

```
## Saved: 06_cox_forest_plot.png
```

```
print(p_cox_forest)
```

```
## `height` was translated to `width`.
```




```
# ============================================================================
# SECTION 6: MULTIPLE TESTING CORRECTION
# ============================================================================
#
# THE MULTIPLE TESTING PROBLEM:
```

```
# If we test 1 hypothesis at p < 0.05, we accept a 5% chance of a false positive.
# If we test 20 hypotheses independently:
#   Expected false positives = 20 × 0.05 = 1
#   Probability of AT LEAST ONE false positive = 1 - (0.95)^20 = 64%!
#
# In immunology/omics, we might test 50 immune markers simultaneously.
# We MUST correct for multiple comparisons.
#
# COMMON METHODS:
```

```r
#   1. Bonferroni: divide  by number of tests. Very conservative.
#      Good for: confirmatory analysis, when we MUST avoid any false positives
#      Bad for: discovery analysis (too many false negatives)
#
#   2. Benjamini-Hochberg (BH) FDR: controls the FALSE DISCOVERY RATE
#      FDR = expected proportion of significant findings that are false positives
#      Good for: discovery analysis (e.g., which of 50 markers are associated?)
#      At FDR = 5%, we accept that 5% of our "hits" might be false - acceptable
#      for generating hypotheses to validate later

cat("\n=== SECTION 6: MULTIPLE TESTING CORRECTION ===\n\n")
```

```
##
## === SECTION 6: MULTIPLE TESTING CORRECTION ===
```

```r
# Simulate testing 50 immune cell markers for association with eGFR
set.seed(42)

n_markers <- 50
marker_names <- paste0("Marker_", sprintf("%02d", 1:n_markers))

# Simulate p-values: most markers are null (no real effect)
# But 8 markers are truly associated (we'll simulate them with lower p-values)
true_associations <- 1:8  # Markers 1-8 have real effects

# For null markers: p-values from Uniform(0,1) - this is what random chance gives
# For true markers: p-values drawn from a distribution skewed toward low values
p_values_raw <- c(
  rbeta(length(true_associations), 0.5, 10),  # True positives: skewed low
  runif(n_markers - length(true_associations), 0, 1)  # Null: uniform
)

# --- Apply corrections ---
p_bonferroni <- p.adjust(p_values_raw, method = "bonferroni")
p_bh_fdr     <- p.adjust(p_values_raw, method = "BH")  # BH = Benjamini-Hochberg

# Compare results
correction_comparison <- data.frame(
  marker         = marker_names,
  p_raw          = round(p_values_raw, 4),
  p_bonferroni   = round(p_bonferroni, 4),
  p_fdr_bh       = round(p_bh_fdr, 4),
  sig_raw        = p_values_raw < 0.05,
  sig_bonferroni = p_bonferroni < 0.05,
  sig_fdr        = p_bh_fdr < 0.05,
  truly_associated = c(rep(TRUE, length(true_associations)),
                       rep(FALSE, n_markers - length(true_associations)))
)

cat("Multiple Testing: Number of significant markers at each threshold:\n")
```

```
## Multiple Testing: Number of significant markers at each threshold:
```

```r
cat(sprintf("  Uncorrected (p < 0.05):          %d significant\n",
            sum(correction_comparison$sig_raw)))
```

```
##   Uncorrected (p < 0.05):          9 significant
```

```
cat(sprintf("  Bonferroni corrected (  = 0.001): %d significant\n",
            sum(correction_comparison$sig_bonferroni)))
```

```
##   Bonferroni corrected (  = 0.001): 2 significant
```

```
cat(sprintf("  BH FDR corrected (FDR < 0.05):   %d significant\n",
            sum(correction_comparison$sig_fdr)))
```

```
##   BH FDR corrected (FDR < 0.05):   2 significant
```

```
cat(sprintf("  True positives (by design):       %d\n",
            length(true_associations)))
```

```
##   True positives (by design):       8
```

```
# Estimate false discovery rates empirically
fp_raw  <- sum(correction_comparison$sig_raw & !correction_comparison$truly_associated)
fp_bon  <- sum(correction_comparison$sig_bonferroni & !correction_comparison$truly_associated)
fp_fdr  <- sum(correction_comparison$sig_fdr & !correction_comparison$truly_associated)

cat(sprintf("\nEstimated false positives:\n"))
```

```
##
## Estimated false positives:
```

```
cat(sprintf("  Uncorrected:  %d false positives out of %d significant\n",
            fp_raw, sum(correction_comparison$sig_raw)))
```

```
##   Uncorrected:  4 false positives out of 9 significant
```

```
cat(sprintf("  Bonferroni:   %d false positives out of %d significant\n",
            fp_bon, sum(correction_comparison$sig_bonferroni)))
```

```
##   Bonferroni:   0 false positives out of 2 significant
```

```
cat(sprintf("  BH FDR:       %d false positives out of %d significant\n",
            fp_fdr, sum(correction_comparison$sig_fdr)))
```

```
##   BH FDR:       0 false positives out of 2 significant
```

```
cat("\nConclusion: BH FDR offers a good balance between sensitivity and specificity\n")
```

```
##
## Conclusion: BH FDR offers a good balance between sensitivity and specificity
```

```
cat("for discovery analyses. Bonferroni is better for confirmatory hypotheses.\n")
```

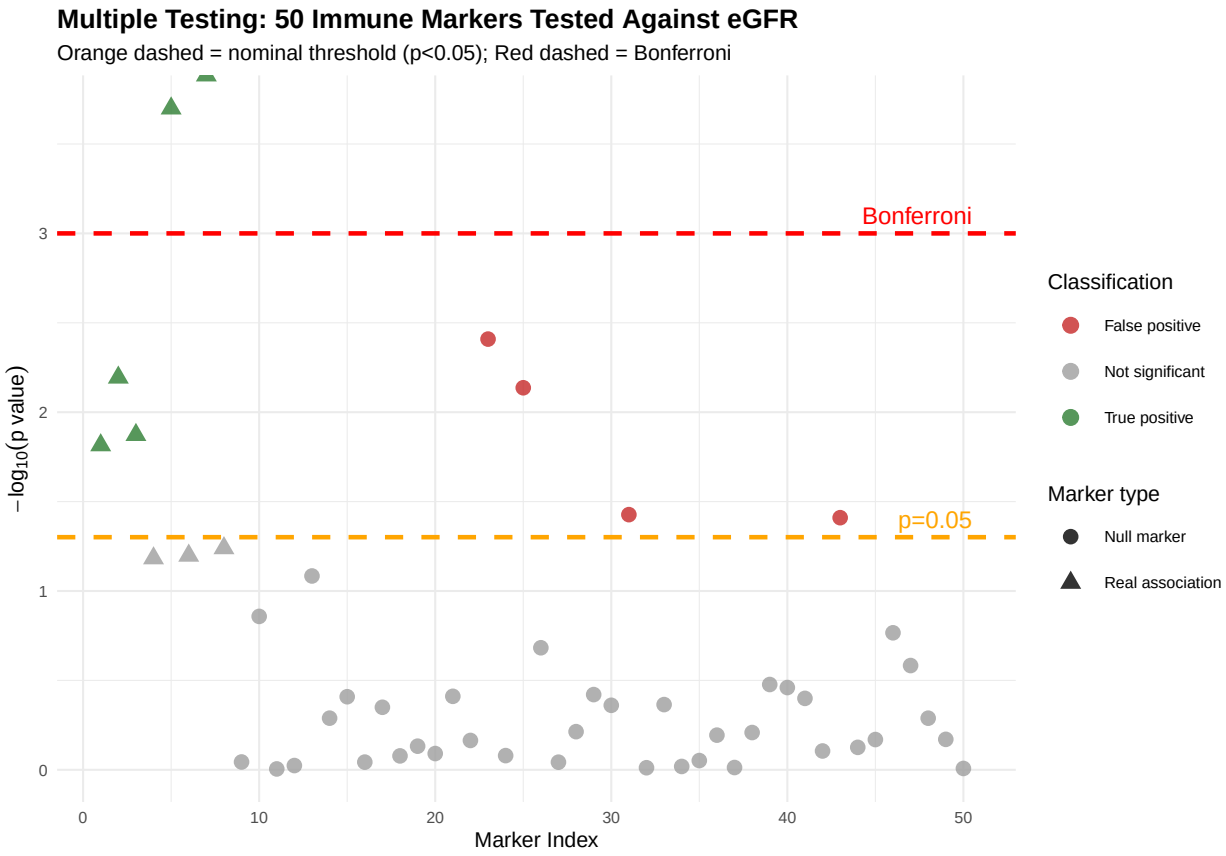
```
## for discovery analyses. Bonferroni is better for confirmatory hypotheses.
```

```
# --- PLOT 7: Multiple Testing Visualisation ---
# Manhattan-style plot: each dot is a marker
# y-axis: -log10(p) so that smaller p-values appear HIGHER
# Horizontal lines: significance thresholds

plot_mt_data <- correction_comparison %>%
  mutate(
    neg_log10_p_raw = -log10(p_raw),
    neg_log10_p_bh  = -log10(p_fdr_bh),
    marker_index    = 1:n()
```

```
  ) %>%
  pivot_longer(
    cols = c(neg_log10_p_raw, neg_log10_p_bh),
    names_to = "correction",
    values_to = "neg_log10_p"
  ) %>%
  mutate(
    correction = recode(correction,
                        "neg_log10_p_raw" = "Uncorrected",
                        "neg_log10_p_bh"  = "BH FDR Corrected"),
    point_color = case_when(
      truly_associated & neg_log10_p > -log10(0.05) ~ "True positive",
      !truly_associated & neg_log10_p > -log10(0.05) ~ "False positive",
      TRUE ~ "Not significant"
    )
  )

p_mt <- ggplot(plot_mt_data %>% filter(correction == "Uncorrected"),
               aes(x = marker_index, y = neg_log10_p, color = point_color,
                   shape = truly_associated)) +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed",
             color = "orange", linewidth = 0.8) +
  geom_hline(yintercept = -log10(0.05 / n_markers), linetype = "dashed",
             color = "red", linewidth = 0.8) +
  geom_point(size = 2.5, alpha = 0.8) +
  annotate("text", x = n_markers + 0.5, y = -log10(0.05) + 0.1,
           label = "p=0.05", size = 3, color = "orange", hjust = 1) +
  annotate("text", x = n_markers + 0.5, y = -log10(0.05 / n_markers) + 0.1,
           label = "Bonferroni", size = 3, color = "red", hjust = 1) +
  scale_color_manual(
    values = c("True positive" = "#2E7D32",
               "False positive" = "#C62828",
               "Not significant" = "#9E9E9E")
  ) +
  scale_shape_manual(values = c("TRUE" = 17, "FALSE" = 16),
                     labels = c("TRUE" = "Real association", "FALSE" = "Null marker")) +
  labs(
    title    = "Multiple Testing: 50 Immune Markers Tested Against eGFR",
    subtitle = "Orange dashed = nominal threshold (p<0.05); Red dashed = Bonferroni",
    x        = "Marker Index",
    y        = expression(-log[10](p~value)),
    color    = "Classification",
    shape    = "Marker type"
  ) +
  theme_minimal(base_size = 8) +
  theme(plot.title = element_text(face = "bold"))

ggsave(file.path(plot_dir, "07_multiple_testing.png"),
       p_mt, width = 10, height = 6, dpi = 300)
cat("\nSaved: 07_multiple_testing.png\n")
```

```
##
## Saved: 07_multiple_testing.png
```

```
print(p_mt)
```




```
# ==============================================================================
# SECTION 7: SUMMARY DASHBOARD PLOT
# ==============================================================================

cat("\n=== Creating summary dashboard plot ===\n")
```

```
##
## === Creating summary dashboard plot ===
```

```
# Simplified versions for the dashboard
p1_mini <- p_egfr_traj +
  labs(title = "A. eGFR Trajectories") +
  theme(plot.subtitle = element_blank(), legend.position = "right",
        plot.caption = element_blank())

p2_mini <- p_mono_violin +
  labs(title = "B. Monocyte Scores") +
  theme(plot.subtitle = element_blank())

p3_mini <- p_scatter +
  labs(title = "C. Monocyte vs eGFR Change") +
  theme(plot.subtitle = element_blank())

p4_mini <- p_cox_forest +
  labs(title = "D. Cox Model HRs") +
```

```
  theme(plot.subtitle = element_blank())

# Combine using patchwork layout
dashboard <- (p1_mini | p2_mini) / (p3_mini | p4_mini) +
  plot_annotation(
    title    = "CKD Cohort: Immune Biomarkers and Disease Progression",
    subtitle = "Module 2 Clinical Biostatistics - Darya Yarparvar",
    theme    = theme(plot.title    = element_text(size = 14, face = "bold"),
                     plot.subtitle = element_text(size = 10, color = "grey40"))
  )

ggsave(file.path(plot_dir, "00_dashboard.png"),
       dashboard, width = 14, height = 10, dpi = 150)
```

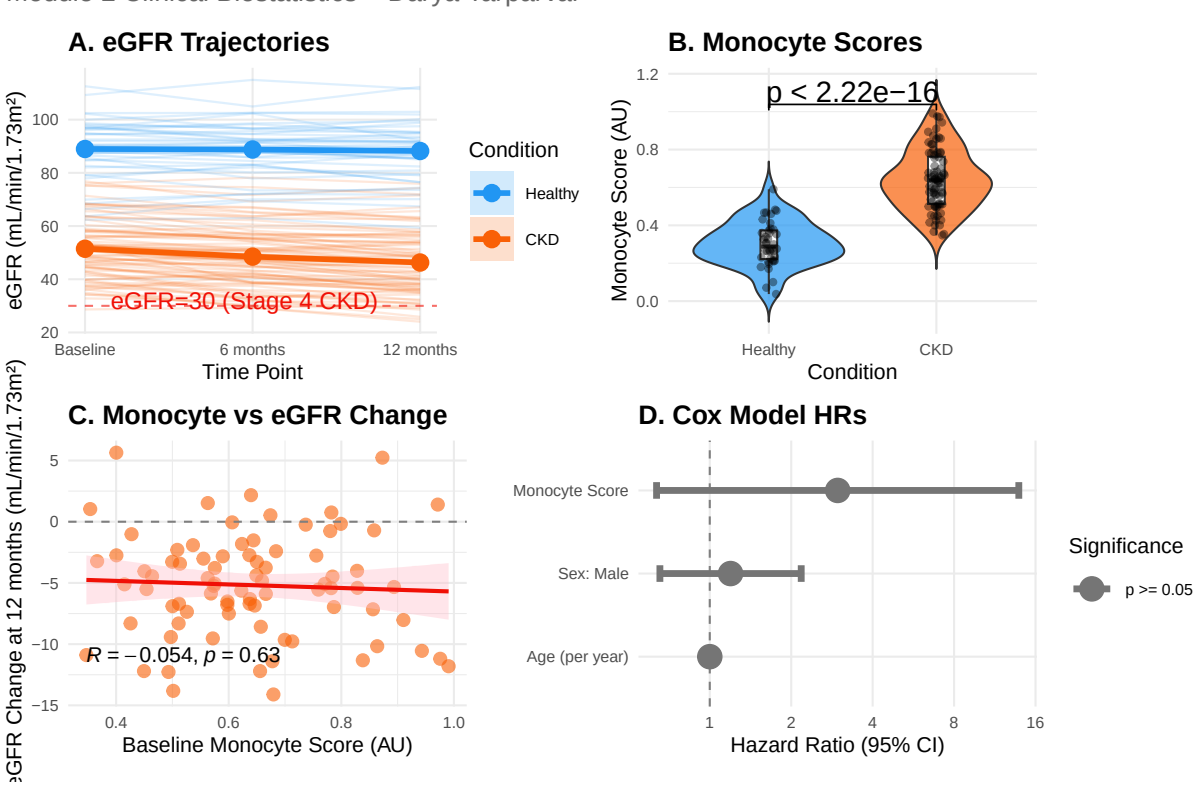
```
## `geom_smooth()` using formula = 'y ~ x'
## `height` was translated to `width`.
```

```
cat("Saved: 00_dashboard.png\n")
```

```
## Saved: 00_dashboard.png
```

```
print(dashboard)
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `height` was translated to `width`.
```




```
# ============================================================================
# SECTION 8: SESSION INFO & REPRODUCIBILITY
```

```r
# ============================================================================
# Always print session info at the end of an analysis script.
# Essential for reproducibility: R version, package versions, OS.

cat("\n=== SESSION INFO ===\n")
```

```
##
## === SESSION INFO ===
```

```r
print(sessionInfo())
```

```
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versio
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK ve
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] ggfortify_0.4.19    ggpubr_0.6.3        lubridate_1.9.5     forcats_1.0.1       stringr_1.5.2
##  [6] purrr_1.2.2         readr_2.1.5         tibble_3.3.0        tidyverse_2.0.0     RColorBrewer_1.1
## [11] scales_1.4.0        patchwork_1.3.2     tidyr_1.3.1         broom.mixed_0.2.9.7 broom_1.0.13
## [16] survival_3.8-3      lmerTest_3.2-1      lme4_2.0-1          Matrix_1.7-4        ggplot2_4.0.0
## [21] dplyr_1.1.4
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1    farver_2.1.2        S7_0.2.0            fastmap_1.2.0       digest_0.6.37
##  [6] timechange_0.4.0    lifecycle_1.0.4     magrittr_2.0.4      compiler_4.5.1      rlang_1.2.0
## [11] tools_4.5.1         utf8_1.2.6          yaml_2.3.10         knitr_1.50          ggsignif_0.6.4
## [16] labeling_0.4.3      abind_1.4-8         withr_3.0.2         numDeriv_2016.8-1.1 grid_4.5.1
## [21] future_1.70.0       globals_0.19.1      MASS_7.3-65         dichromat_2.0-0.1   tinytex_0.57
## [26] cli_3.6.5           rmarkdown_2.30      crayon_1.5.3        ragg_1.5.0          reformulas_0.4.4
## [31] generics_0.1.4      rstudioapi_0.17.1   tzdb_0.5.0          minqa_1.2.8         splines_4.5.1
## [36] parallel_4.5.1      vctrs_0.7.3         boot_1.3-32         carData_3.0-6       car_3.1-5
## [41] hms_1.1.4           rstatix_0.7.3       Formula_1.2-5       listenv_0.10.1      systemfonts_1.3.
## [46] glue_1.8.0          parallelly_1.47.0   nloptr_2.2.1        codetools_0.2-20    stringi_1.8.7
## [51] gtable_0.3.6        furrr_0.4.0         pillar_1.11.1       htmltools_0.5.8.1   R6_2.6.1
## [56] textshaping_1.0.4   Rdpack_2.6.6        evaluate_1.0.5      lattice_0.22-7      rbibutils_2.4.1
## [61] backports_1.5.0     Rcpp_1.1.0          gridExtra_2.3       nlme_3.1-168        mgcv_1.9-3
## [66] xfun_0.53           pkgconfig_2.0.3
```

```r
# ============================================================================
# EXERCISES
# ============================================================================
#
```

```r
# EXERCISE 1 (Easy): Descriptive Statistics
#   Add a column `eGFR_stage` to baseline_data that categorises patients:
#     Stage 3a: eGFR 45-59
#     Stage 3b: eGFR 30-44
#     Stage 4: eGFR 15-29
#     Stage 5: eGFR < 15
#   Then create a table counting patients per stage by condition.
#
# EXERCISE 2 (Medium): Mixed Model Extension
#   Add a random slope for time to model2: change (1|patient_id) to
#   (1 + time_months|patient_id). Does this improve model fit?
#   Compare with anova(model2, model2_with_slope).
#   HINT: Use REML=FALSE for model comparison with anova().
#
# EXERCISE 3 (Medium): Survival Analysis
#   Stratify the KM curves by monocyte score tertile (low/mid/high).
#   HINT: Use ntile(monocyte_score, 3) from dplyr to create tertiles.
#   Do high-monocyte patients fail faster?
#
# EXERCISE 4 (Hard): Time-Varying Covariates
#   In Cox models, we used baseline monocyte score. But it was measured at
#   3 time points. Use a time-varying Cox model (coxph with counting process
#   formulation) to incorporate monocyte score measured at each visit.
#   HINT: Search for "counting process Cox model R" and Surv(start, stop, event)
#
# EXERCISE 5 (Hard): Mediation Analysis
#   Hypothesis: CKD → elevated monocyte score → faster eGFR decline
#   Is the effect of CKD on eGFR mediated by monocyte score?
#   Use the mediation package and mediate() function.
#   HINT: This requires fitting two models (mediator model + outcome model)
#
# EXERCISE 6 (Conceptual): Assumptions
#   For Model 1 (lmer), check:
#     1. Normality of residuals: plot(density(resid(model1)))
#     2. Homoscedasticity: plot(fitted(model1), resid(model1))
#     3. Normality of random effects: ranef(model1)$patient_id %>% qqnorm()
#   What would we do if these assumptions were violated?
#
# EXERCISE 7 (Applied): Power Analysis
#   Before designing a CKD study, we need to estimate required sample size.
#   For a mixed model with 3 time points and expected interaction  = -0.15:
#   How many patients do we need to achieve 80% power?
#   HINT: Use the simr package: extend() and powerSim()
#
# ================================================================================
# END OF MODULE 2
# ================================================================================

cat("\n\n=== MODULE 2 COMPLETE ===\n")
```

```
##
##
## === MODULE 2 COMPLETE ===
```

```r
cat("Plots saved to:", plot_dir, "\n")
```

```
## Plots saved to: module2_plots
```

```r
cat("Files created:\n")
```

```
## Files created:
```

```r
for (f in list.files(plot_dir, pattern = "*.png")) {
  cat("  ", file.path(plot_dir, f), "\n")
}
```

```
##    module2_plots/00_dashboard.png
##    module2_plots/01_eGFR_trajectories.png
##    module2_plots/02_immune_scores_violin.png
##    module2_plots/03_monocyte_vs_egfr_change.png
##    module2_plots/04_lmer_coefficients.png
##    module2_plots/05_kaplan_meier_auto.png
##    module2_plots/05_kaplan_meier.png
##    module2_plots/06_cox_forest_plot.png
##    module2_plots/07_multiple_testing.png
```

```r
# ==============================================================================
```